APPENDIX A

Setting Up Rust and Lean

This appendix explains how to install the two toolchains the book uses, how to set up an editor that shows Lean's goals as you work, how to run the code and the verification tools, and how to rebuild the book itself.

In this appendix you will learn:

- how to install Rust with rustup and Lean with elan, and how the repository pins the exact versions;
- how to download the prebuilt Mathlib library;
- how to set up VS Code for Rust and Lean, and how to type Lean's symbols;
- how to run the examples, check the proofs and verify the whole book;
- how to rebuild the Markdown chapters and the PDFs.

A.1 The repository

Clone the repository, or download and unpack it, anywhere you like. All the commands in this appendix are run from the repository root unless a `cd` says otherwise.

```
book/              the chapters
code/
├── rust/          Cargo workspace (Rust 1.99.0, standard library only)
└── lean/          Lake project (Lean 4.34.1 with Mathlib)
tools/             build and verification scripts
```

A.2 Rust

Install Rust with **rustup**, the official toolchain manager. On macOS and Linux:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

On Windows, download and run `rustup-init.exe` from https://rustup.rs; it will also ask you to install the Visual Studio C++ build tools if they are missing.

You do not need to choose a Rust version. The file `code/rust/rust-toolchain.toml` pins the compiler the book was written with:

```
# The book shows real compiler diagnostics, so the exact compiler is pinned.
# rustup installs it automatically the first time you run cargo here.
```

```
# rust-src is the standard library's source, which rust-analyzer needs.
[toolchain]
channel = "1.99.0"
components = ["rustfmt", "rust-src"]
```

The first time you run `cargo` inside `code/rust`, rustup notices the file and downloads Rust 1.99.0 (and `rustfmt`) automatically. Everywhere else your default toolchain is used. The version is pinned because the book shows real compiler diagnostics, and their wording changes between releases. Newer compilers accept the same programs; only some messages may read differently.

Check the installation:

```
cd code/rust
cargo run --example ch01_first_types
```

The workspace has no external dependencies, so this builds in seconds.

A.3 Lean and Mathlib

Install Lean with **elan**, Lean's toolchain manager (the counterpart of rustup). On macOS and Linux:

```
curl https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh -sSf | sh
```

On Windows, and for other options, follow the instructions on https://lean-lang.org. Installing the Lean 4 extension for VS Code (see below) also offers to install elan for you.

Again, you do not need to choose a version. The file `code/lean/lean-toolchain` contains a single line, `leanprover/lean4:v4.34.1`, and elan downloads that version the first time you run `lean` or `lake` in the directory.

Lake is Lean's build tool and package manager, the counterpart of Cargo. The project's configuration is `code/lean/lakefile.toml`:

```
# The book's Lean project. It depends on Mathlib, pinned to the release that
# matches the toolchain in lean-toolchain. Before the first build, download the
# prebuilt library (under 1 GB to download, about 8 GB on disk) with
#   lake exe cache get
# otherwise Lake would compile Mathlib from source, which takes hours.
name = "book"
version = "0.1.0"
defaultTargets = ["Book"]

[[require]]
name = "mathlib"
git = "https://github.com/leanprover-community/mathlib4"
rev = "v4.34.1"

[[lean_lib]]
name = "Book"
globs = ["Book.+"]
```

It declares one library, `Book`, whose modules are the files under `code/lean/Book/` (the module `Book.Ch05.Kripke` is the file `Book/Ch05/Kripke.lean`), and one dependency, Mathlib, pinned to the release that matches the toolchain. The exact revisions of Mathlib and of its own dependencies are recorded in `lake-manifest.json`, the counterpart of `Cargo.lock`.

Mathlib is large: compiling it from source takes hours. Its maintainers publish prebuilt files for every release, and a command provided by Mathlib downloads them. Run it once, before the first build:

```
cd code/lean
lake exe cache get
```

The download is under a gigabyte of compressed files, which take about 8 GB once unpacked (with the sources of Mathlib and its dependencies) in `code/lean/.lake/`; the compressed files are also kept in a cache in your home directory so that other projects using the same Mathlib release can share it. Then check every Lean file of the book:

```
lake build
```

A build after the first one only rechecks the files that changed. You can also check a single file, which prints every message Lean produces for it:

```
lake env lean Book/Ch01/FirstSteps.lean
```

`lake env` runs a command with the project's search paths set, so that `import Mathlib...` finds the library.

> **WARNING**
>
> If Lean reports `unknown module prefix 'Mathlib'`, or starts compiling hundreds of Mathlib files, the prebuilt library is missing or does not match. Run `lake exe cache get` again in `code/lean`. Never run `lake update` in the book's project: it would move Mathlib to a newer revision than the one the book was checked with.

A.4 An editor for Lean

You can read the Lean files in any editor, but to *work* with Lean you want an editor that runs Lean as you type. The standard choice is **VS Code** with the **Lean 4** extension (published by `leanprover`). For Rust, add the **rust-analyzer** extension.

Open the folder `code/lean` itself in VS Code (not the repository root), so that the extension finds `lakefile.toml` and the toolchain. Open a file such as `Book/Ch01/FirstSteps.lean`. After a moment the extension starts Lean, and a panel called the **infoview** appears on the right. The infoview is the most important tool for learning Lean:

- With the cursor on a command such as `#eval` or `#check`, it shows the output.
- With the cursor inside a proof, it shows the **goal**: the hypotheses available at that point (above the turnstile `⊢`) and what remains to be proved (below it). Moving the cursor from tactic to tactic shows how each step changes the goal. This is how the comments in the book's proofs were written, and it is the best way to read them.
- With the cursor on an error (underlined in red), it shows the full message.

Hovering over a name shows its type and documentation; Ctrl-click (Cmd-click on macOS) jumps to its definition, including definitions inside Lean's core library and Mathlib. Reading the source of a definition is often the quickest way to understand it.

Typing symbols

Lean code uses Unicode symbols such as `→`, `∀`, `ℕ` and `⟨ ⟩`. In VS Code you type them with a backslash followed by a name, and the extension replaces the sequence as you type. A few you will use constantly:

Symbol	Type	Symbol	Type
`→`	`\to`	`ℕ`	`\N`
`∀`	`\forall`	`ℤ`	`\Z`
`∃`	`\exists`	`⟨ ⟩`	`\<` and `\>`
`∧`	`\and`	`≤`	`\le`
`∨`	`\or`	`≠`	`\ne`
`¬`	`\not`	`↔`	`\iff`
`λ`	`\lambda`	`⊢`	`\vdash`
`α β γ`	`\alpha` …	`∘`	`\circ`

Most LaTeX names work (`\Gamma`, `\tau`, `\otimes`, `\times`). When you meet a symbol you do not know how to type, hover over it in the editor: the tooltip shows the input sequence.

A.5 Running and verifying the code

The Rust examples are run with Cargo from `code/rust`. Single-file examples are in `code/rust/examples/`; examples that use one of the book's crates live in that crate's `examples/` directory and need `-p`:

```
cd code/rust
cargo run --example ch05_kripke
cargo run -p stlc --example ch08_stlc
cargo test -p affine
```

The book shows the output of every example. Those outputs are stored as snapshots in `code/rust/output/`, and the programs that must be *rejected* by the compiler are in `code/rust/compile_fail/`, each with a `.stderr` file holding the diagnostics the book shows. A script checks all of it, together with every Lean file:

```
python3 tools/run_examples.py                # everything
python3 tools/run_examples.py --only ch22    # one chapter
python3 tools/run_examples.py --rust --tests # Rust examples and the crates' tests
python3 tools/run_examples.py --lean         # Lean only
```

For each Rust example it compiles the program (failing on any warning), runs it and compares its output with the snapshot. For each compile-fail program it checks that the compiler rejects it with the expected error codes and compares the diagnostics with the `.stderr` file. For Lean it runs `lake build` and fails if Lean prints any message at all: every output shown in the book is wrapped in `#guard_msgs`,

which checks it against the comment above the command. When you change an example on purpose, `--bless` rewrites the snapshots from the real output:

```
python3 tools/run_examples.py --only ch22 --bless
```

The script needs Python 3.11 or newer and no extra packages.

A.6 Building the book

The chapters are written in Markdown in `book/src/`, with `{{#include}}` directives that pull code, outputs and diagnostics from `code/`. The build script expands them into self-contained Markdown files in `book/chapters/`, renders the diagrams, and produces a PDF for each chapter and one for the whole book:

```
python3 tools/build_book.py check    # validate includes, anchors, math and images
python3 tools/build_book.py all      # diagrams, chapters/*.md, PDFs, README toc
python3 tools/build_book.py pdf --only 08 --no-book   # one chapter's PDF
```

It requires pandoc 3.x, Typst 0.13 or newer and Graphviz (`dot`). On macOS all three are available from Homebrew (`brew install pandoc typst graphviz`); on Linux, from your distribution or the projects' release pages. The conventions for writing chapters, examples and proofs are in `tools/AUTHORING.md`.

A.7 Summary

- rustup and elan install the toolchains; `rust-toolchain.toml` and `lean-toolchain` pin Rust 1.99.0 and Lean 4.34.1, and are honored automatically.
- Run `lake exe cache get` once in `code/lean` to download prebuilt Mathlib; never run `lake update` in the book's project.
- Work on Lean files in VS Code with the Lean 4 extension and keep the infoview open; type symbols with backslash sequences.
- `cargo run --example ...` runs an example, `lake build` checks the proofs, and `tools/run_examples.py` verifies everything the book shows.
- `tools/build_book.py all` rebuilds the chapters and PDFs.